

React JS Ecosystem

Complete End-to-End Developer Guide

Why React, SPA vs MPA, Client vs Server Rendering, React vs Angular,
Hooks, Router, Redux Toolkit, Auth/JWT, npm, Vite, DevTools, HTTP/CORS,
Production Deployment — Deep Concepts, Real Examples, Bugs & Interview Q&A

Table of Contents

1. Why React Came — Life Before React
2. SPA vs MPA Deep Dive
3. Client-Side Rendering vs Server-Side Rendering vs Static Generation
4. React vs Angular vs Vue — Full Comparison
5. Why React Is Considered Better Than Angular (For Many Teams)
6. Creating a React Project — CRA vs Vite
7. React Project Folder Structure Deep Dive
8. React Execution Flow — How React Actually Works
9. Virtual DOM & Reconciliation Deep Dive
10. JSX Deep Dive
11. Components — Functional vs Class
12. Props Deep Dive
13. State Deep Dive
14. Component Lifecycle — Class Components vs Hooks Equivalent
15. Conditional Rendering
16. Lists & Keys Deep Dive
17. Forms in React — Controlled vs Uncontrolled
18. Event Handling in React
19. Composition vs Inheritance
20. Context API Deep Dive
21. Error Boundaries
22. React.memo & Performance Optimization
23. Fragments & Portals
24. Higher-Order Components & Render Props
25. React Hooks — Introduction & Rules of Hooks
26. useState Deep Dive
27. useEffect Deep Dive
28. useContext Deep Dive
29. useRef Deep Dive
30. useMemo & useCallback Deep Dive

31. useReducer Deep Dive
32. Custom Hooks Deep Dive
33. React Router — Why Routing Is Needed in an SPA
34. React Router Deep Dive — Routes, Link, useNavigate, useParams
35. Nested Routes & Protected Routes
36. REST API Integration in React
37. Axios vs Fetch Deep Dive
38. Axios Interceptors & Centralized API Client
39. Redux Toolkit — Why State Management Is Needed
40. Redux Toolkit Deep Dive — Store, Slice, Reducers, Actions
41. useSelector & useDispatch
42. Async Thunks & API Calls in Redux Toolkit
43. Authentication & JWT Deep Dive
44. Storing Tokens & Protected Routes in React
45. Refresh Token Flow in React
46. npm Deep Dive
47. package.json & Semantic Versioning
48. Vite Deep Dive — Why Vite Over Webpack/CRA
49. Vite Configuration & Hot Module Replacement
50. Browser DevTools Deep Dive
51. Debugging React with React DevTools
52. HTTP & HTTPS Deep Dive
53. CORS Deep Dive
54. Production Deployment — Where & How Frontend Apps Are Deployed
55. Deployment Deep Dive — Nginx, Docker, Vercel/Netlify, Azure, S3+CloudFront
56. CI/CD for React Apps
57. Environment Variables & Config Per Environment
58. Common React Bugs, Issues & Fixes (Extended)
59. Common Production Issues & Fixes
60. React Interview Questions & Answers (40 Q&A)
61. Quick Revision Summary

1. Why React Came — Life Before React

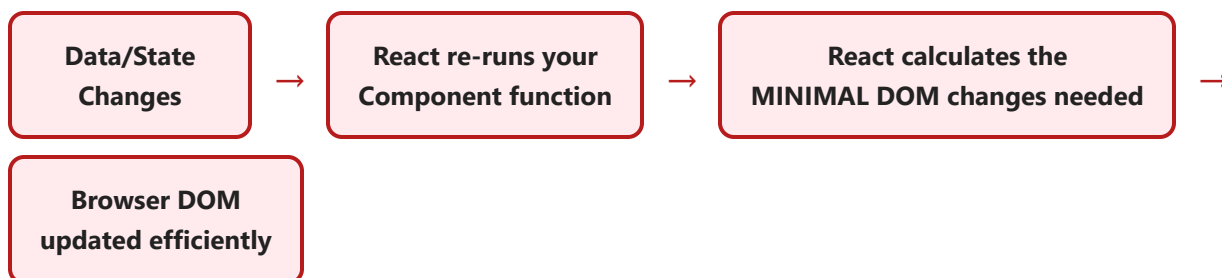
Before React (released by Facebook in 2013), web developers manually manipulated the DOM using plain JavaScript or jQuery — every time data changed, you had to write imperative code telling the browser EXACTLY which elements to find and update.

```
// Life before React – manual, imperative DOM manipulation with jQuery
$("#cart-count").text(cartItems.length);
$("#cart-total").text("$" + calculateTotal(cartItems));
if (cartItems.length === 0) {
  $("#empty-cart-message").show();
} else {
  $("#empty-cart-message").hide();
}
// Every single piece of UI that depends on "cartItems" needs its OWN manual update code,
// scattered across the codebase – easy to forget one, causing the UI to go out of sync with
// data.
```

What problems did this cause in real, growing applications?

Problem	Real Impact
UI and data going out of sync	A developer updates data but forgets one of the five places the UI needed to reflect it — a real, extremely common bug class
No clear structure for complex UIs	jQuery code sprawls into deeply nested callbacks and manual DOM queries scattered across many files
Manual DOM manipulation is slow at scale	Directly touching the real DOM for every small change is computationally expensive on large, complex pages
No reusable "component" concept	Copy-pasting similar HTML/JS blocks for similar UI pieces (a product card repeated 50 times) instead of a true reusable unit

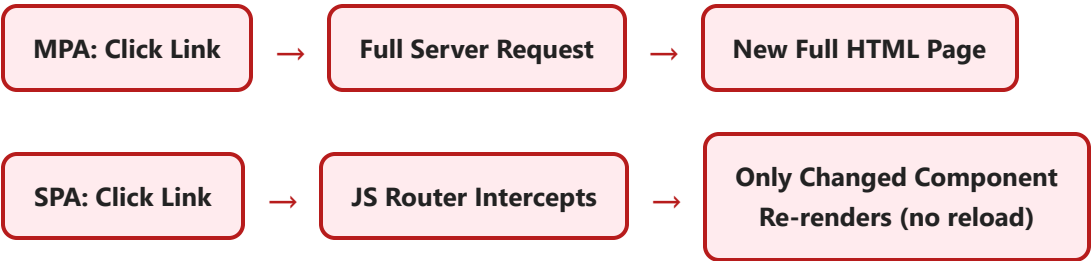
React solved this with a **declarative** model: instead of telling the browser HOW to update the DOM step by step, you describe WHAT the UI should look like for a given piece of data (state), and React figures out the most efficient way to update the actual DOM to match.



Real-time example: A cart icon showing item count, a cart total, and an "empty cart" message all automatically stay in sync in React by simply reading from the SAME `cartItems` state — change the state once, and every piece of UI depending on it re-renders correctly, with zero manual "don't forget to also update X" bookkeeping.

2. SPA vs MPA Deep Dive

Aspect	MPA (Multi-Page Application)	SPA (Single-Page Application)
Navigation	Every link triggers a FULL page reload — a new HTML document is fetched from the server	The browser loads ONE HTML shell once; JavaScript swaps content in place, without a full reload
Perceived speed after first load	Slower — every navigation re-downloads and re-renders the entire page from scratch	Much faster — only the data/components that actually change are updated
Initial load	Fast — server sends ready-to-display HTML immediately	Can be slower — browser must download and execute a JavaScript bundle before showing meaningful content (mitigated by SSR, Section 3)
SEO (out of the box)	Naturally strong — search engines receive fully-formed HTML immediately	Historically weaker for pure client-rendered SPAs — solved today via Server-Side Rendering frameworks (Next.js) when SEO matters
Typical use case	Content-heavy sites, blogs, traditional e-commerce catalog pages	Interactive dashboards, admin panels, social media feeds, real-time applications



Real-time example: Gmail is a textbook SPA — clicking between "Inbox", "Sent", and an individual email never triggers a visible full-page reload; only the email list/content area updates, while the surrounding sidebar and header remain untouched, giving it an "app-like" feel rather than a "website" feel.

3. Client-Side Rendering vs Server-Side Rendering vs Static Generation

Approach	How It Works	Trade-off
CSR (Client-Side Rendering)	Server sends a near-empty HTML shell + a JS bundle; the BROWSER runs React to build the actual page content	Simple to deploy (just static files), but slower "time to first meaningful content" and weaker default SEO
SSR (Server-Side Rendering)	The SERVER runs React to generate the full HTML for that specific request, sending ready-to-display content immediately; React then "hydrates" it client-side to make it interactive	Faster initial paint and strong SEO, but requires a running Node.js server (or serverless functions) rather than pure static hosting
SSG (Static Site Generation)	HTML is generated ONCE at BUILD time for every page, then served as plain static files forever after	Extremely fast and cheap to host (pure static files/CDN), but content is only as fresh as the last build — not ideal for highly dynamic, per-user data

CSR: Browser Renders

SSR: Server Renders Per-Request

SSG: Pre-built at Compile Time

Hydration is the process where React "takes over" server-rendered HTML in the browser, attaching event listeners and making the already-visible markup interactive — without hydration, SSR HTML would look right but do nothing when clicked.

Real-time example: A Next.js e-commerce site uses SSG for rarely-changing marketing pages (About, FAQ), SSR for product pages needing fresh, per-request stock/pricing data, and pure CSR for an authenticated "My Orders" dashboard that doesn't need to be indexed by search engines at all — all three rendering strategies coexisting in ONE application, chosen per page based on its actual needs.

4. React vs Angular vs Vue — Full Comparison

Aspect	React	Angular	Vue
Type	A library (just the view layer)	A full, opinionated framework	A progressive framework
Language	JavaScript/JSX (TypeScript optional but common)	TypeScript (mandatory)	JavaScript (TypeScript optional)
Learning curve	Moderate — core concepts are small, ecosystem choices (router, state) are separate decisions	Steep — many built-in concepts (DI, modules, RxJS, decorators) to learn upfront	Gentle — often considered the easiest to pick up initially
Data binding	One-way (explicit state updates trigger re-render)	Two-way (by default, via ngModel)	Two-way (via v-model), but supports one-way too
Bundle size / performance	Small core, flexible — can be very lean with careful choices	Larger out of the box due to its full-framework nature	Small, comparable to React
Flexibility vs opinionation	Highly flexible — YOU choose routing, state management, form libraries	Highly opinionated — Angular ships its own router, forms, HTTP client, DI system built in	Balanced — sensible defaults with room to swap parts out
Backed by	Meta (Facebook)	Google	Independent open-source (Evan You)

5. Why React Is Considered Better Than Angular (For Many Teams)

Important framing: "Better" is genuinely context-dependent — this section explains the REASONS many teams prefer React, not an absolute universal truth. Angular remains an excellent, productive choice for large enterprise teams wanting an all-in-one, strongly-opinionated framework.

Reason Teams Prefer React	Explanation
Gentler learning curve	React's core API surface (components, props, state, a handful of hooks) is small; Angular requires learning Dependency Injection, RxJS Observables, decorators, and NgModules just to build a basic app
Flexibility of choice	Teams pick exactly the router/state-management/form library that fits their needs (or none at all for simple apps), rather than being locked into Angular's built-in, more rigid equivalents
Larger ecosystem & hiring pool	React has historically had the largest developer community and job market share among the three, making hiring and finding community solutions easier
JSX feels closer to plain JavaScript	Many developers find JSX (HTML-like syntax directly in JS) more intuitive than Angular's separate template syntax with its own directives (*ngIf, *ngFor) and binding syntax to learn
Simpler mental model for updates	One-way data flow (explicit state → re-render) is often easier to reason about and debug than Angular's two-way binding, where changes can originate from multiple places at once
Performance tuning control	React's smaller core lets teams add exactly the state/data-fetching libraries they need, avoiding Angular's comparatively heavier default bundle size for simple apps

Real-time example: A startup building an MVP dashboard chose React specifically because new hires (often coming from varied backgrounds) could become productive within days learning just components/props/state/hooks, versus the weeks some team members reported needing to become comfortable with Angular's full DI/RxJS/module system on a previous project.

6. Creating a React Project — CRA vs Vite

```
# Legacy approach – Create React App (CRA), now considered outdated/unmaintained by many teams
npx create-react-app my-app
cd my-app
npm start

# Modern, recommended approach – Vite (Section 47-48 covers WHY in depth)
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Tool	Status
Create React App (CRA)	The original official tool for years; now effectively unmaintained and noticeably slower — no longer the recommended starting point for new projects
Vite	The current community-standard choice — dramatically faster dev server startup and Hot Module Replacement (Section 48)
Next.js	The standard choice specifically when SSR/SSG (Section 3) is needed, or for full-stack React applications

7. React Project Folder Structure Deep Dive

```

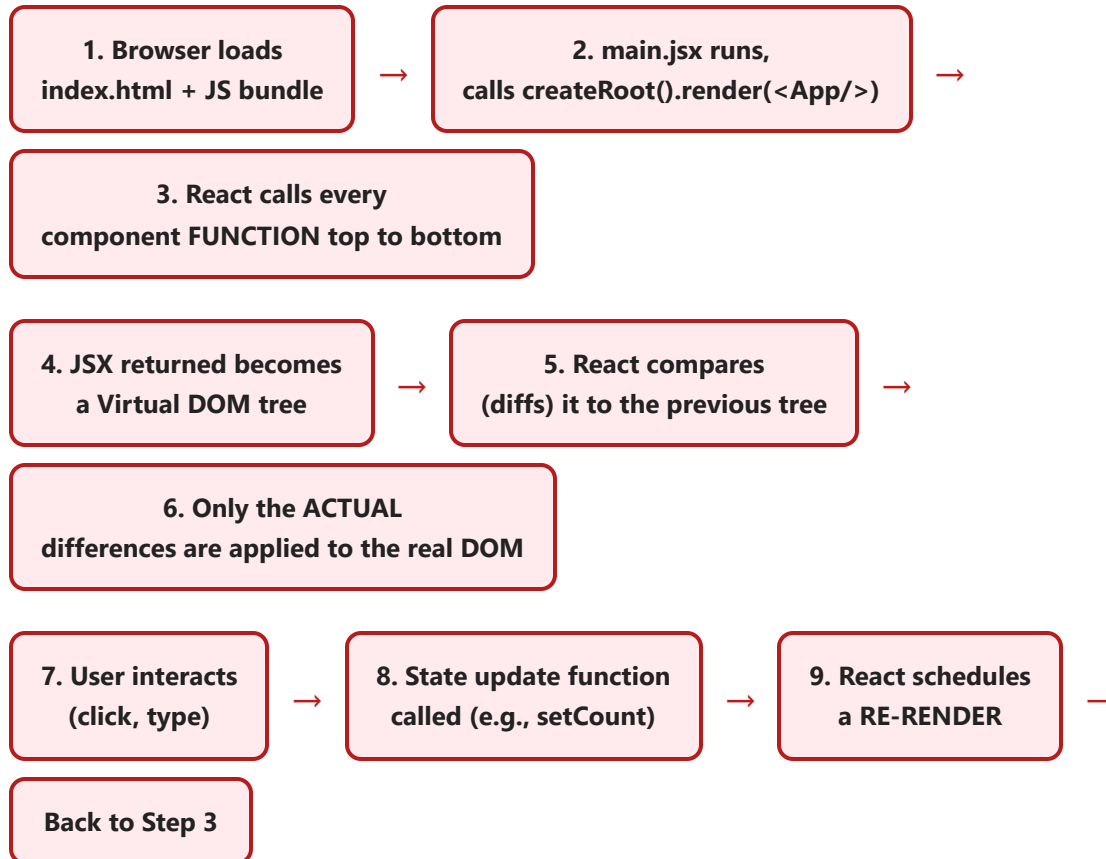
my-app/
├── public/
│   └── index.html      <- the single real HTML shell; React mounts into #root
├── src/
│   ├── main.jsx        <- entry point – renders <App /> into the DOM
│   ├── App.jsx         <- the root component, usually holds routing
│   ├── components/     <- small, reusable UI pieces (Button, Card, Modal)
│   │   ├── Button.jsx
│   │   └── Button.css
│   ├── pages/          <- one component per ROUTE/page (HomePage, LoginPage)
│   ├── hooks/          <- custom hooks (Section 32)
│   ├── context/        <- React Context providers (Section 20)
│   ├── store/          <- Redux Toolkit slices/store (Section 38-41)
│   ├── services/ (or api/) <- centralized Axios/fetch API call functions
│   ├── utils/          <- pure helper functions (formatting, validation)
│   └── assets/         <- images, fonts, static files
├── .env                <- environment variables (Section 56)
├── package.json
└── vite.config.js

```

Folder	Why Separated
components/ vs pages/	Distinguishes small, REUSABLE UI pieces from full page-level compositions tied to a specific route — a Button is reused everywhere; a LoginPage exists exactly once
services/ or api/	Centralizes every backend API call in one place, so components don't scatter raw fetch/axios calls throughout the codebase, similar in spirit to a Service layer in Spring Boot
hooks/	Holds custom hooks — reusable STATEFUL logic (Section 32) extracted out of components

No single "official" structure exists — this is the most common, battle-tested convention across real teams. What matters most is **CONSISTENCY** across the team, and separating "reusable UI", "page-level composition", "API calls", and "business logic" into clearly distinct locations.

8. React Execution Flow — How React Actually Works

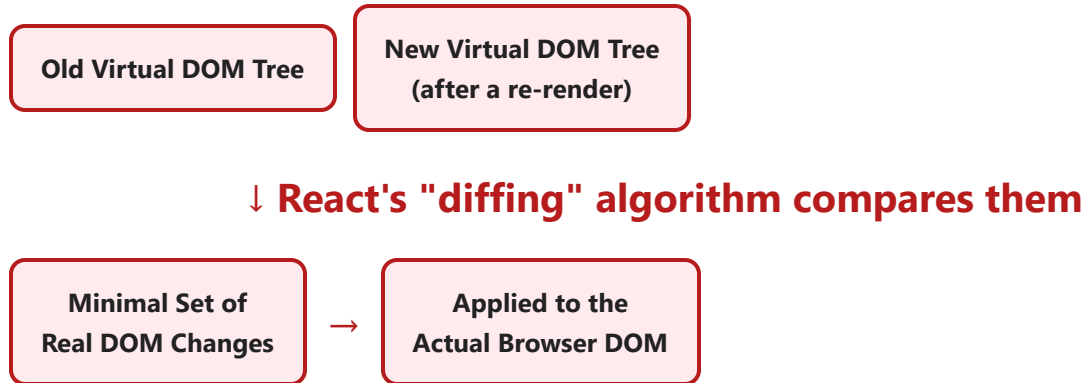


In simple terms: React components are just JavaScript FUNCTIONS that describe "what the UI should look like right now, given this data." Every time relevant data (state/props) changes, React calls that function again, gets a new description, and efficiently patches only what actually changed in the real browser DOM.

Why this matters: A React component function should ideally be a PURE function of its props/state (same inputs, same JSX output) — this predictability is exactly what makes React's re-render model reliable and is a direct real-world application of the "pure function" concept from the JavaScript guide's functional programming section.

9. Virtual DOM & Reconciliation Deep Dive

The real browser DOM is SLOW to update directly, especially for large, frequent changes — every real DOM mutation can trigger expensive layout recalculation (the "Critical Rendering Path" from the HTML/CSS guide). React's **Virtual DOM** is a lightweight, in-memory JavaScript representation of the UI that React can compare/update MUCH faster than touching the real DOM directly.



Reconciliation is the specific name for this diffing-and-patching process. React uses a heuristic algorithm (not a full, expensive tree-diff) that makes reasonable assumptions — for example, it assumes elements of a DIFFERENT type at the same position represent a genuinely different thing and will fully replace them rather than trying to patch them, which is why stable, correct `key` props (Section 16) matter so much for list reconciliation specifically.

Real-time example: Updating ONE todo item's text in a list of 500 todos only causes React to patch that SINGLE corresponding real DOM node — the other 499 are left completely untouched, since the Virtual DOM diff correctly identifies only one node actually changed.

10. JSX Deep Dive

```
function Greeting({ name }) {  
  return (  
    <div className="greeting">           {/* className, NOT class – "class" is a reserved  
JS word */}  
      <h1>Hello, {name}!</h1>           {/* {} embeds a real JS expression directly */}  
      {name === "Admin" && <span>★</span>}  {/* conditional rendering using && */}  
    </div>  
  );  
}
```

JSX is NOT HTML — it's syntax sugar that Babel/the build tool compiles down to plain

`React.createElement(...)` function calls. This is why JSX requires `className` instead of `class`, `htmlFor` instead of `for`, and camelCase event names (`onClick`) — these are just JavaScript function call arguments under the hood, following JS naming conventions, not real HTML attribute names.

```
// What JSX actually compiles to:  
const element = <h1 className="title">Hello</h1>;  
// becomes:  
const element = React.createElement("h1", { className: "title" }, "Hello");
```

ISSUE "Adjacent JSX elements must be wrapped in an enclosing tag" compile error.

FIX A component's `return` can only return ONE root element — wrap multiple sibling elements in a single parent `<div>` or a Fragment `<>...</>` (Section 23), which adds no extra DOM node.

11. Components — Functional vs Class

```
// Class component (the ORIGINAL way, largely legacy in new code today)
class Greeting extends React.Component {
  render() { return <h1>Hello, {this.props.name}</h1>; }
}

// Functional component with Hooks (the MODERN standard since 2019)
function Greeting({ name }) {
  return <h1>Hello, {name}</h1>;
}
```

Aspect	Class Components	Functional Components (with Hooks)
Syntax	Verbose — extends React.Component, render() method, "this" everywhere	Concise — just a plain JavaScript function
State	this.state + this.setState()	useState hook (Section 26)
Lifecycle	Named methods (componentDidMount, etc. — Section 14)	useEffect hook covers all lifecycle needs in one unified API
Reusing stateful logic	Awkward (Higher-Order Components/render props, Section 24)	Simple — Custom Hooks (Section 32)
Current recommendation	Legacy — only seen in older codebases	The standard for ALL new React code today

12. Props Deep Dive

```
function ProductCard({ title, price, onAddToCart }) { // props destructured directly in the
  parameter
  return (
    <div>
      <h3>{title}</h3>
      <p>${price}</p>
      <button onClick={() => onAddToCart(title)}>Add to Cart</button>
    </div>
  );
}
// Usage – passing data DOWN from parent to child
<ProductCard title="Running Shoe" price={79.99} onAddToCart={handleAddToCart} />
```

Props flow in ONE direction only — from parent to child. A child component can never directly modify its own props (they are read-only); if a child needs to trigger a change, the PARENT passes down a callback function (like `onAddToCart` above) that the child calls, and the parent decides how to actually update its own state in response.

ISSUE "Cannot assign to read only property" or unexpected behavior when a component tries to directly modify a prop it received.

FIX Props are immutable from the child's perspective by design — lift the value into the PARENT's own state instead, and pass both the value and an update callback down as props.

13. State Deep Dive

```
function Counter() {  
  const [count, setCount] = useState(0); // [currentValue, updaterFunction]  
  
  function handleClick() {  
    setCount(count + 1); // triggers a re-render with the new value  
  }  
  return <button onClick={handleClick}>Clicked {count} times</button>;  
}
```

State is local and private to a component by default (unlike props, which come from outside). Calling a state SETTER function (like `setCount`) is what actually tells React "re-render this component" — directly mutating a state variable (`count++`) does NOT trigger a re-render and is a common beginner bug, directly connecting to the immutability principle from the JavaScript guide.

ISSUE Calling `setCount(count + 1)` twice in a row inside the same event handler only increments the count by 1, not 2.

FIX State updates within one event handler are BATCHED and both reads see the same, stale "count" value — use the functional updater form instead: `setCount(prev => prev + 1)`; called twice correctly increments by 2, since each call receives the truly latest pending value.

14. Component Lifecycle — Class Components vs Hooks Equivalent

Lifecycle Moment	Class Component Method	Hooks Equivalent
Component first appears on screen	<code>componentDidMount()</code>	<code>useEffect(() => {...}, [])</code> — empty dependency array
Component re-renders after a specific value changes	<code>componentDidUpdate(prevProps, prevState)</code>	<code>useEffect(() => {...}, [someValue])</code> — runs when someValue changes
Component is about to be removed from the page	<code>componentWillUnmount()</code>	The function RETURNED from inside <code>useEffect</code> (the "cleanup function")

```
useEffect(() => {  
  const timer = setInterval(() => console.log("tick"), 1000); // like componentDidMount  
  return () => clearInterval(timer); // like componentWillUnmount - cleanup!  
}, []); // empty array = run once on mount, cleanup once on unmount
```

Hooks unify all three lifecycle "moments" into ONE consistent mental model (`useEffect` + its dependency array + its optional cleanup function), rather than three separate, differently-named methods — widely considered one of the biggest usability improvements Hooks brought over class components.

15. Conditional Rendering

```
function UserGreeting({ user }) {  
  if (!user) return <p>Please log in.</p>;           // early return pattern  
  
  return (  
    <div>  
      {user.isAdmin && <AdminPanel />}             {/* && – render only if true */}  
      {user.isPremium ? <PremiumBadge /> : <UpgradeLink />} {/* ternary for either/or */}  
    </div>  
  );  
}
```

ISSUE `{items.length && <ItemList items={items} />}` renders a stray "0" on screen when the list is empty.

FIX When `items.length` is `0` (falsy but not literally false), JSX renders the number 0 itself rather than nothing — use an explicit boolean comparison instead: `{items.length > 0 && <ItemList items={items} />}`.

16. Lists & Keys Deep Dive

```
function TodoList({ todos }) {  
  return (  
    <ul>  
      {todos.map(todo => (  
        <li key={todo.id}>{todo.text}</li>    /* key must be STABLE and UNIQUE per item  
*/  
      ))}  
    </ul>  
  );  
}
```

Why the key prop is critical: React uses `key` to match items between re-renders during reconciliation (Section 9) — without a stable key tied to the actual DATA (never the array INDEX, if the list can reorder/filter/insert), React can misidentify which DOM element corresponds to which item, causing bugs like form input values "sticking" to the wrong row after a reorder.

ISSUE Using the array index as the key causes input field values or component state to get "mixed up" after items are reordered, filtered, or deleted from the middle of a list.

FIX Use a stable, unique property from the actual data (like a database ID) as the key instead of the array index — index-based keys only work safely for lists that NEVER reorder/filter/insert in the middle.

17. Forms in React — Controlled vs Uncontrolled

```
// Controlled – React state is the SINGLE SOURCE OF TRUTH for the input's value (the standard approach)
function LoginForm() {
  const [email, setEmail] = useState("");
  return <input value={email} onChange={(e) => setEmail(e.target.value)} />;
}

// Uncontrolled – the DOM itself holds the value; React reads it only when needed, via a ref
function LoginForm2() {
  const emailRef = useRef();
  function handleSubmit() { console.log(emailRef.current.value); }
  return <input ref={emailRef} />;
}
```

Approach	When to Use
Controlled	The default, recommended approach — needed whenever you want instant validation, conditional formatting, or to keep the input in sync with other UI as the user types
Uncontrolled	Simple forms, file inputs (which can't be controlled by React at all), or integrating with non-React code/libraries expecting direct DOM access

18. Event Handling in React

```
function Button() {  
  function handleClick(event) {  
    event.preventDefault();    // React wraps the native event in a "SyntheticEvent" –  
    same API, cross-browser consistent  
    console.log("Clicked!", event.target);  
  }  
  return <button onClick={handleClick}>Click Me</button>;  // camelCase event name, a  
  function REFERENCE, not a call  
}
```

ISSUE A click handler fires IMMEDIATELY on render instead of waiting for an actual click.

FIX Writing `onClick={handleClick()}` CALLS the function immediately during render; use `onClick={handleClick}` (a reference) or `onClick={() => handleClick(someArg)}` when you need to pass arguments.

19. Composition vs Inheritance

```
// React strongly favors COMPOSITION over class inheritance  
function Card({ children }) {    // "children" is a special prop – whatever's  
  nested inside  
    return <div className="card">{children}</div>;  
}  
// Usage – composing different content INTO the same reusable Card shell  
<Card>  
  <h2>Title</h2>  
  <p>Some content here.</p>  
</Card>
```

The official React documentation explicitly recommends composition (building complex UIs by combining simpler components, using patterns like `children` or passing components as props) over building deep class inheritance hierarchies — inheritance-based UI hierarchies tend to become rigid and hard to extend as an app grows.

20. Context API Deep Dive

```
const ThemeContext = createContext("light"); // create a context with a default value

function App() {
  return (
    <ThemeContext.Provider value="dark">      {/* provides the value to ALL descendants */}
      <Toolbar />
    </ThemeContext.Provider>
  );
}
function Toolbar() { return <ThemedButton />; } // doesn't need to know/pass "theme" at all!
function ThemedButton() {
  const theme = useContext(ThemeContext);      // reads the value directly, however
  deeply nested
  return <button className={theme}>Click</button>;
}
```

The problem Context solves — "prop drilling": Without Context, passing a value (like the current theme or logged-in user) through 5 layers of components that don't actually use it themselves — only to hand it to a deeply nested child that does — clutters every intermediate component's props. Context lets any descendant read a value directly, skipping the intermediate layers entirely.

When NOT to use Context: For frequently-changing, large-scale application state (like a big shopping cart or complex form), Context can cause EVERY consuming component to re-render on any change — Redux Toolkit (Section 38) or more targeted state tools are usually a better fit at that scale.

21. Error Boundaries

```
class ErrorBoundary extends React.Component {           // MUST be a class component – no Hooks
  equivalent yet
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(error, info) { console.error("Caught:", error, info); }
  render() {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
// Wrap any part of the tree that might throw during rendering
<ErrorBoundary>
  <ProductList />
</ErrorBoundary>
```

An Error Boundary catches JavaScript errors thrown DURING RENDERING anywhere in its child tree, showing a fallback UI instead of a blank white screen/crashed app — it does NOT catch errors in event handlers, async code, or server-side rendering, which still need regular try/catch handling.

22. React.memo & Performance Optimization

```
const ProductCard = React.memo(function ProductCard({ title, price }) {  
  console.log("Rendering:", title);  
  return <div>{title} - ${price}</div>;  
});  
// Now ProductCard only re-renders if its OWN props (title, price) actually changed –  
// it will SKIP re-rendering if its parent re-renders for an unrelated reason.
```

`React.memo` wraps a component to skip re-rendering when its props haven't changed (using a shallow comparison) — valuable specifically for components that are expensive to render AND re-render often due to an unrelated parent update, but unnecessary (and adds a small comparison overhead) for cheap, simple components.

ISSUE A component wrapped in `React.memo` still re-renders every single time, seemingly making no difference.

FIX A NEW object/array/function is very likely being passed as a prop on every parent render (e.g., an inline arrow function or object literal), which fails `React.memo`'s shallow equality check every time — wrap such callback props in `useCallback` (Section 30) in the parent.

23. Fragments & Portals

```
function TableRow() {
  return (
    <>                                /* Fragment shorthand – groups elements WITHOUT adding an
extra DOM node */
    <td>Name</td>
    <td>Age</td>
    </>
  );
}

// Portal – renders children into a DIFFERENT DOM node, outside the normal component tree
// hierarchy
function Modal({ children }) {
  return ReactDOM.createPortal(children, document.getElementById("modal-root"));
}
```

Fragments avoid unnecessary wrapper `<div>`s that could break valid HTML structure (e.g., a component returning multiple `<td>` elements must NOT wrap them in a `<div>` inside a `<table>`). Portals are essential for modals/tooltips/dropdowns that need to visually escape a parent with `overflow:hidden` or a constrained z-index stacking context (HTML/CSS guide, Section 27), while still behaving like a normal React child for event bubbling purposes.

24. Higher-Order Components & Render Props

```
// Higher-Order Component (HOC) – a function that takes a component and returns an ENHANCED
component
function withLoading(WrappedComponent) {
  return function(props) {
    if (props.isLoading) return <p>Loading...</p>;
    return <WrappedComponent {...props} />;
  };
}
const ProductListWithLoading = withLoading(ProductList);

// Render Props – a component that takes a FUNCTION as a prop, calling it to determine what to
render
function DataFetcher({ render }) {
  const [data, setData] = useState(null);
  useEffect(() => { fetchData().then(setData); }, []);
  return render(data);
}
<DataFetcher render={({data} => data ? <List items={data} /> : <Spinner /> } />
```

Largely superseded by Custom Hooks today: Both patterns exist to REUSE stateful logic across components — before Hooks (2019), they were the only real options. Custom Hooks (Section 32) now solve the exact same problem far more simply, so HOCs/render props are mostly seen in older codebases or specific library APIs today, but remain a common interview topic.

25. React Hooks — Introduction & Rules of Hooks

Hooks (introduced in React 16.8, 2019) let functional components use state, lifecycle behavior, and other React features that were previously only possible in class components — enabling the "functional components with Hooks" style that is now the standard.

The Two Rules of Hooks (strictly enforced by an ESLint plugin)

1. **Only call Hooks at the TOP LEVEL** — never inside loops, conditions, or nested functions. React relies on Hooks being called in the EXACT SAME ORDER on every render to correctly associate each hook call with its own internal state.
2. **Only call Hooks from React function components or custom Hooks** — never from regular JavaScript functions or class components.

ISSUE

"React Hook is called conditionally" ESLint error, or state becomes mysteriously mismatched/incorrect after a code change.

FIX

A Hook was placed inside an `if` statement or after an early `return` — always call every Hook unconditionally at the top of the component, and put any conditional LOGIC inside the hook's callback/body instead.

26. useState Deep Dive

```
const [user, setUser] = useState(null);           // lazy initial value option:
const [count, setCount] = useState(() => computeExpensiveInitialValue()); // function form –
runs ONCE only

setUser({ name: "Alex" });                        // direct value
setCount(prevCount => prevCount + 1);              // functional updater – always safe, uses
the LATEST value

// Updating an OBJECT in state correctly (immutably) – never mutate directly!
setUser(prevUser => ({ ...prevUser, name: "New Name" }));
```

Always prefer the FUNCTIONAL updater form (`setCount(prev => prev + 1)`) whenever the new state depends on the previous state — it avoids subtle bugs from stale closures capturing an outdated value, especially inside event handlers or effects that run after some delay (Section 13's batching issue).

27. useEffect Deep Dive

```
useEffect(() => {
  console.log("Runs after EVERY render");
});

useEffect(() => {
  console.log("Runs ONCE, after the first render only");
}, []); // empty dependency array

useEffect(() => {
  console.log("Runs after first render AND whenever 'userId' changes");
  fetchUser(userId).then(setUser);
}, [userId]); // dependency array with a value

useEffect(() => {
  const subscription = subscribeToChat(roomId);
  return () => subscription.unsubscribe(); // cleanup – runs before the NEXT effect, and on
  unmount
}, [roomId]);
```

Dependency Array	When the Effect Runs
Omitted entirely	After every single render — rarely what you actually want
<code>[]</code> (empty)	Only once, after the very first render (like <code>componentDidMount</code>)
<code>[someValue]</code>	After the first render, AND again whenever <code>someValue</code> changes between renders

ISSUE "React Hook `useEffect` has a missing dependency" ESLint warning, or an effect uses a STALE value from an old render.

FIX Every value from component scope used **INSIDE** the effect must be listed in the dependency array — omitting one means the effect's closure captures an outdated value from whenever it was first created; add it to the array (or restructure the logic) rather than suppressing the lint rule.

ISSUE An API call inside `useEffect` fires repeatedly in an infinite loop.

FIX A **NEW** object/array/function is being created inline and included in the dependency array — since it's a new reference every render, the effect re-triggers every render, which triggers a state update, which re-renders, forever; memoize the dependency with `useMemo/useCallback`, or restructure to depend on primitive values instead.

28. useContext Deep Dive

```
const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = async (credentials) => { /* ... */ setUser(result); };
  return (
    <AuthContext.Provider value={{ user, login }}>
      {children}
    </AuthContext.Provider>
  );
}

function useAuth() { return useContext(AuthContext); } // a small custom hook wrapper is a
common convention

// Any deeply nested component:
function ProfileButton() {
  const { user, login } = useAuth();
  return user ? <span>{user.name}</span> : <button onClick={login}>Log In</button>;
}
```

29. useRef Deep Dive

```
function TextInputWithFocusButton() {
  const inputRef = useRef(null);
  function handleClick() { inputRef.current.focus(); } // direct DOM node access
  return (<><input ref={inputRef} /><button onClick={handleClick}>Focus Input</button></>);
}

function Timer() {
  const countRef = useRef(0);
  function increment() {
    countRef.current++; // updates INSTANTLY, but does NOT trigger a re-render
    console.log(countRef.current);
  }
}
```

useRef	useState
Changing <code>.current</code> does NOT trigger a re-render	Calling the setter DOES trigger a re-render
Value persists across renders, mutated directly	Value persists across renders, replaced immutably
Used for: DOM access, storing a mutable value that shouldn't cause re-renders (timer IDs, previous values)	Used for: any value that should visually update the UI when it changes

30. useMemo & useCallback Deep Dive

```
const expensiveResult = useMemo(() => {
  return calculateExpensiveTotal(items); // only RE-RUNS when 'items' actually
}, [items]);                             changes

const handleClick = useCallback(() => {
  doSomething(id);                       // returns the SAME function reference
}, [id]);                                // unless 'id' changes – useful with
React.memo (Section 22)
```

Hook	Memoizes	Primary Use Case
useMemo	A computed VALUE	Avoiding re-running an expensive calculation on every render
useCallback	A FUNCTION reference itself	Keeping a stable function reference so a React.memo-wrapped child doesn't needlessly re-render, or so a useEffect dependency doesn't change every render

Don't overuse these: Both add their own small overhead (comparing dependencies every render) — reach for them specifically to fix a MEASURED performance problem (a genuinely expensive calculation, or an observed unnecessary re-render chain), not defensively on every value/function in every component.

31. useReducer Deep Dive

```
function cartReducer(state, action) {
  switch (action.type) {
    case "ADD_ITEM": return [...state, action.payload];
    case "REMOVE_ITEM": return state.filter(item => item.id !== action.payload.id);
    case "CLEAR": return [];
    default: return state;
  }
}

function Cart() {
  const [cart, dispatch] = useReducer(cartReducer, []);
  function addItem(item) { dispatch({ type: "ADD_ITEM", payload: item }); }
  return <button onClick={() => addItem({id:1, name:"Shoe"})}>Add</button>;
}
```

useState vs useReducer: useState is simplest for independent, simple values. useReducer is preferred once state logic involves MULTIPLE related sub-values updated together via distinct "actions" (exactly like a mini-Redux, Section 38, pattern) — it centralizes all the state-transition logic in one reducer function instead of scattering many separate setState calls throughout event handlers.

32. Custom Hooks Deep Dive

```
function useFetch(url) { // a custom hook – just a function starting
  with "use"
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
      setLoading(true);
      fetch(url)
        .then(res => res.json())
        .then(setData)
        .catch(setError)
        .finally(() => setLoading(false));
    }, [url]);

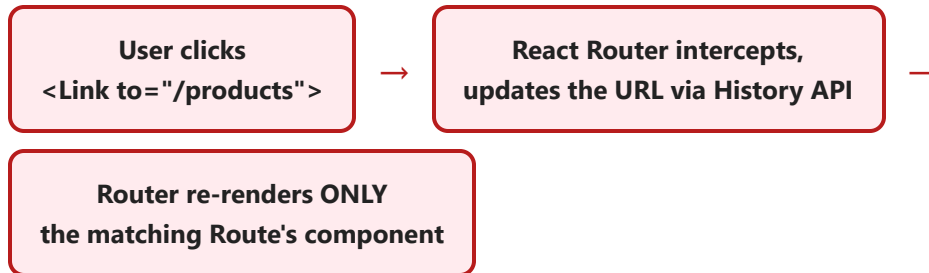
    return { data, loading, error };
  }

  // Now ANY component can reuse this entire stateful data-fetching pattern in ONE line:
  function ProductPage() {
    const { data: products, loading, error } = useFetch("/api/products");
    if (loading) return <Spinner />;
    if (error) return <ErrorMessage error={error} />;
    return <ProductList products={products} />;
  }
}
```

Custom Hooks are THE modern way to extract and reuse stateful logic across components — directly replacing the older Higher-Order Component and Render Props patterns (Section 24) with something far simpler to read, write, and compose.

33. React Router — Why Routing Is Needed in an SPA

Since an SPA (Section 2) never triggers real page reloads, the BROWSER's native URL navigation doesn't naturally "just work" — clicking a link would need to somehow update the visible content and the URL bar without a real page reload. **React Router** is the standard library solving exactly this: mapping URL paths to which component tree should render, all client-side, using the browser's History API under the hood.



34. React Router Deep Dive — Routes, Link, useNavigate, useParams

```

import { BrowserRouter, Routes, Route, Link, useNavigate, useParams } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/products">Products</Link>
      </nav>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/products" element={<ProductListPage />} />
        <Route path="/products/:id" element={<ProductDetailPage />} />    {/* :id is a
URL parameter */}
        <Route path="*" element={<NotFoundPage />} />                    {/* catch-
all 404 */}
      </Routes>
    </BrowserRouter>
  );
}

function ProductDetailPage() {
  const { id } = useParams();          // reads ":id" from the current URL, e.g. "42"
  const navigate = useNavigate();
  function goBack() { navigate(-1); } // programmatic navigation (like browser "back")
  return <div>Showing product {id}</div>;
}

```

API	Purpose
<Link>	Renders a real, accessible <a> tag, but intercepts the click to navigate WITHOUT a full page reload
useNavigate()	Programmatic navigation from inside event handlers/effects (e.g., redirecting after a successful login)
useParams()	Reads dynamic URL segments (like a product ID) declared with <code>:paramName</code> in the route path

ISSUE Using a plain `<a href="/products">` tag instead of `<Link>` causes a full, jarring page reload in what should be a smooth SPA navigation.

FIX A native anchor tag triggers the browser's default full-navigation behavior, bypassing React Router entirely — always use `<Link>` (or `useNavigate`) for in-app navigation within a React Router-managed SPA.

35. Nested Routes & Protected Routes

```
// Nested routes – a shared layout wrapping multiple child routes
<Routes>
  <Route path="/dashboard" element={<DashboardLayout />>
    <Route index element={<DashboardHome />} />
    <Route path="settings" element={<DashboardSettings />} />
  </Route>
</Routes>
// DashboardLayout renders its own nav/sidebar, plus <Outlet /> where the matched child route
appears

// Protected Route – a wrapper component enforcing authentication before rendering its children
function ProtectedRoute({ children }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  return children;
}
<Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
```

Critical security reminder: A frontend "Protected Route" is a UX convenience ONLY — it hides a page visually/in-app, but a determined user can still call the underlying API directly (via Postman/curl). The BACKEND must independently enforce real authorization on every API endpoint (Spring Security, covered in the dedicated Spring Security guide) — frontend route guards are never a real security boundary on their own.

36. REST API Integration in React

```
useEffect(() => {
  let cancelled = false;
  async function loadProducts() {
    try {
      const res = await fetch("/api/products");
      const data = await res.json();
      if (!cancelled) setProducts(data);
    } catch (err) {
      if (!cancelled) setError(err);
    }
  }
  loadProducts();
  return () => { cancelled = true; }; // avoid setting state on an unmounted component
}, []);
```

This pattern — fetch on mount, handle loading/error/success, and guard against updating state after the component unmounts — is exactly what the custom `useFetch` hook (Section 32) or a dedicated data-fetching library (React Query/TanStack Query) exists to abstract away, since writing this boilerplate correctly by hand in every component is repetitive and error-prone.

37. Axios vs Fetch Deep Dive

Feature	fetch (native)	Axios (library)
JSON parsing	Manual — <code>await res.json()</code>	Automatic — <code>response.data</code> is already parsed
Error on HTTP error status (404/500)	No — must manually check <code>response.ok</code>	Yes — automatically throws/rejects for non-2xx responses
Interceptors	Not built in	Built in (Section 37) — a major reason teams choose Axios
Request cancellation	<code>AbortController</code> (manual)	<code>AbortController</code> support, plus its own older <code>CancelToken</code> API
Bundle size	Zero — built into every browser	An added dependency, though small

```
// Axios example
import axios from "axios";
const res = await axios.get("/api/products");
console.log(res.data);           // already parsed JSON, no .json() call needed

try {
  await axios.post("/api/orders", orderData);
} catch (error) {
  console.log(error.response.status);    // Axios automatically throws for 4xx/5xx
}
```

38. Axios Interceptors & Centralized API Client

```
// api/client.js – ONE shared, configured instance used everywhere in the app
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL, timeout: 10000 });

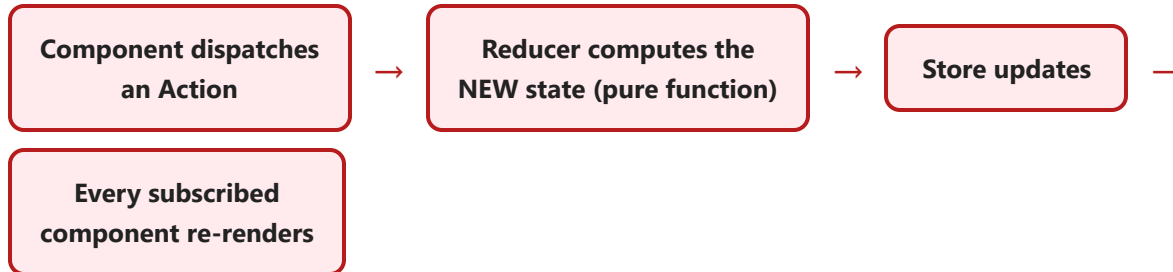
api.interceptors.request.use(config => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`; // attach auth to EVERY
  request automatically
  return config;
});

api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) { // global 401 handling in
ONE place
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);
export default api;
```

Interceptors let you handle cross-cutting API concerns (auth token attachment, global 401 redirect, logging, retry logic) in exactly ONE place, instead of repeating the same boilerplate in every single component that makes an API call — directly analogous to a Spring Boot filter/interceptor on the backend.

39. Redux Toolkit — Why State Management Is Needed

As an app grows, more and more UNRELATED components need access to the SAME shared data (the logged-in user, a shopping cart, notification counts) — passing this through props alone ("prop drilling") or overusing Context (Section 20, which can cause broad re-renders) becomes unwieldy. **Redux** provides ONE centralized, predictable store for shared application state, with a strict, traceable pattern for how it can change.



Redux Toolkit (RTK) is the officially recommended, modern way to use Redux — it eliminates the large amount of repetitive boilerplate (action type constants, action creators, manual immutable update logic) that classic/legacy Redux required.

40. Redux Toolkit Deep Dive — Store, Slice, Reducers, Actions

```
// features/cart/cartSlice.js
import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [] },
  reducers: {
    addItem: (state, action) => {
      state.items.push(action.payload);    // looks like mutation, but RTK uses Immer
internally – safe!
    },
    removeItem: (state, action) => {
      state.items = state.items.filter(item => item.id !== action.payload);
    }
  }
});
export const { addItem, removeItem } = cartSlice.actions;    // auto-generated action creators
export default cartSlice.reducer;

// store.js
import { configureStore } from "@reduxjs/toolkit";
export const store = configureStore({
  reducer: { cart: cartSlice.reducer }
});

// main.jsx
import { Provider } from "react-redux";
<Provider store={store}><App /></Provider>
```

The "mutation that isn't": Redux Toolkit uses a library called Immer internally, letting you write reducer code that LOOKS like direct mutation (`state.items.push(...)`) while Immer actually produces a correct, brand-new immutable state object behind the scenes — this was a deliberate design choice to remove classic Redux's most tedious, error-prone boilerplate (manual spread-based immutable updates).

41. useSelector & useDispatch

```
import { useSelector, useDispatch } from "react-redux";
import { addItem, removeItem } from "../cartSlice";

function CartSummary() {
  const items = useSelector(state => state.cart.items); // reads a SLICE of the store,
  subscribes to its changes
  const dispatch = useDispatch();

  function handleRemove(id) {
    dispatch(removeItem(id)); // sends an action to the store
  }
  return <p>{items.length} items in cart</p>;
}
```

`useSelector` automatically re-renders the component ONLY when the specific piece of state it selects actually changes — selecting a narrow, specific slice (`state.cart.items`) rather than the whole store avoids unnecessary re-renders when unrelated parts of the store update.

42. Async Thunks & API Calls in Redux Toolkit

```
import { createAsyncThunk, createSlice } from "@reduxjs/toolkit";

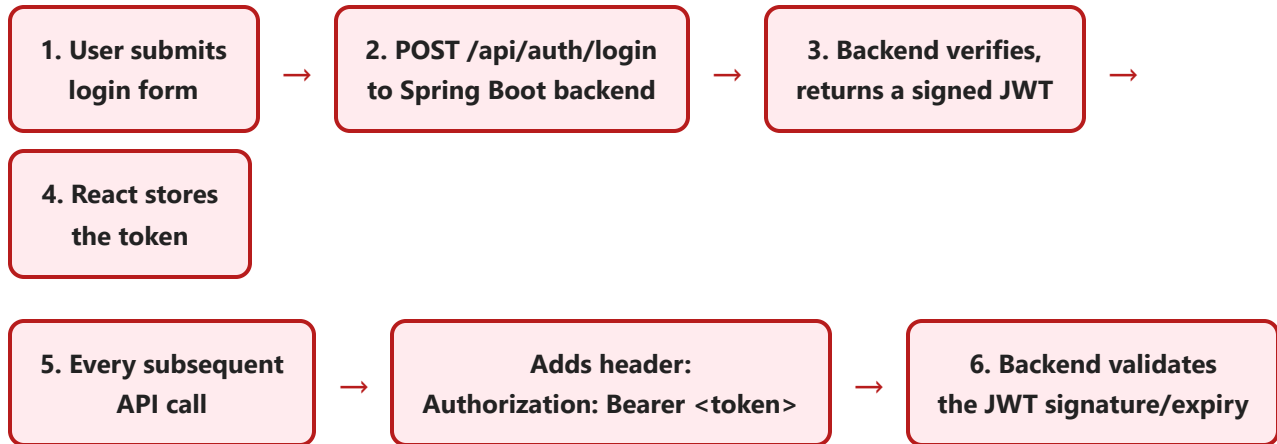
export const fetchProducts = createAsyncThunk("products/fetch", async () => {
  const res = await fetch("/api/products");
  return res.json();
});

const productsSlice = createSlice({
  name: "products",
  initialState: { items: [], status: "idle" },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchProducts.pending, (state) => { state.status = "loading"; })
      .addCase(fetchProducts.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.items = action.payload;
      })
      .addCase(fetchProducts.rejected, (state) => { state.status = "failed"; });
  }
});

// Usage in a component:
useEffect(() => { dispatch(fetchProducts()); }, [dispatch]);
```

`createAsyncThunk` automatically generates `pending` / `fulfilled` / `rejected` action types for an async operation, letting the reducer cleanly handle loading/success/error states — the standard Redux Toolkit pattern for any API call whose result needs to live in the global store.

43. Authentication & JWT Deep Dive



```
async function login(email, password) {  
  const res = await api.post("/auth/login", { email, password });  
  localStorage.setItem("token", res.data.token); // or store in memory/Context –  
  Section 44  
  setUser(res.data.user);  
}
```

A JWT (JSON Web Token) is a self-contained, cryptographically SIGNED token carrying claims (user ID, roles, expiry) — the backend can verify it wasn't tampered with using just the signature, without needing a database lookup on every single request, exactly as covered in the dedicated Spring Security guide's JWT implementation sections.

44. Storing Tokens & Protected Routes in React

Storage Option	Trade-off
localStorage	Simple, persists across browser restarts, but readable by ANY JavaScript on the page — vulnerable if an XSS attack occurs anywhere in the app
In-memory (React state/Context only)	Safer against XSS token theft since it's not persisted anywhere accessible to injected scripts, but lost on page refresh (requiring a re-login or silent refresh flow)
HttpOnly Cookie (set by the backend)	Not readable by JavaScript at all (immune to XSS token theft), but requires CSRF protection and correct cross-origin cookie configuration

There's a genuine, actively-debated security trade-off here — many real production teams accept localStorage's risk for simplicity, while security-sensitive applications (banking) often invest in HttpOnly cookie-based flows instead, exactly mirroring the same trade-off discussion in the Spring Security guide.

45. Refresh Token Flow in React

```
api.interceptors.response.use(
  response => response,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true; // prevent infinite
      retry loops
      const { data } = await axios.post("/auth/refresh", { refreshToken: getRefreshToken() });
    });
    setToken(data.accessToken);
    error.config.headers.Authorization = `Bearer ${data.accessToken}`;
    return api(error.config); // retry the original
    failed request
  }
  return Promise.reject(error);
});
```

This interceptor-based pattern automatically refreshes an expired access token and SILENTLY retries the original failed request — the user experiences no interruption at all, rather than being abruptly logged out the moment a short-lived access token expires.

46. npm Deep Dive

```
npm install          # installs every dependency listed in package.json
npm install axios    # adds a new dependency + updates package.json automatically
npm install --save-dev vitest # adds to devDependencies (build/test tools, not needed in
production)
npm uninstall axios
npm run dev          # runs the "dev" script defined in package.json
npm ci               # a STRICTER install, using package-lock.json EXACTLY – standard
for CI pipelines
```

`npm ci` (Clean Install) deletes `node_modules` first and installs EXACTLY what's pinned in `package-lock.json`, failing if `package.json` and the lock file are out of sync — this strictness is exactly why CI/CD pipelines use `npm ci` instead of `npm install`, guaranteeing a reproducible, consistent build every time.

47. package.json & Semantic Versioning

```
{
  "dependencies": {
    "react": "^18.2.0",    // ^ = accepts 18.x.x (any minor/patch update, but NOT 19.0.0)
    "axios": "~1.4.0"     // ~ = accepts 1.4.x only (patch updates only, not even minor)
  }
}
```

Version Part	Meaning (Semantic Versioning: MAJOR.MINOR.PATCH)
MAJOR	Breaking changes — upgrading requires reviewing your code for compatibility
MINOR	New features added, backward compatible — safe to upgrade without code changes
PATCH	Bug fixes only, fully backward compatible

48. Vite Deep Dive — Why Vite Over Webpack/CRA

Aspect	Older Tools (Webpack/CRA)	Vite
Dev server startup	Slow — bundles the ENTIRE app before serving anything, even for a huge app	Near-instant — serves source files directly over native browser ES modules, no upfront bundling needed
Hot Module Replacement (HMR)	Can become noticeably slower as the app grows larger	Stays fast regardless of app size, since only the CHANGED module needs updating, not a re-bundle
Production build	Webpack	Uses Rollup under the hood for a highly optimized production bundle

Vite achieves its fast dev-server speed by leveraging the fact that modern browsers natively support ES Modules (`import` / `export` , JavaScript guide Section 39) — it doesn't need to bundle everything upfront for DEVELOPMENT, only transforming/serving files on demand as the browser actually requests them.

49. Vite Configuration & Hot Module Replacement

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    proxy: {
      '/api': 'http://localhost:8080' // forwards API calls to the Spring Boot backend
    }
  }
});
```

HMR (Hot Module Replacement) updates changed modules IN PLACE in the running browser, preserving current component state (e.g., form input values, open modals) instead of a full page reload — dramatically speeding up the development feedback loop, especially for deeply nested UI states that are tedious to re-create after every change.

Real-time example: The `server.proxy` config above lets a React dev server running on port 3000 transparently forward any `/api/**` request to a local Spring Boot backend on port 8080 — completely avoiding CORS issues (Section 53) during local development, since the browser only ever talks to port 3000.

50. Browser DevTools Deep Dive

DevTools Tab	What It's For
Elements	Inspect/edit the live DOM and computed CSS styles — see exactly which CSS rule is winning the cascade
Console	View <code>console.log</code> output, JavaScript errors/warnings, and run ad-hoc JS directly against the live page
Network	Inspect every HTTP request/response — headers, status codes, timing, payload — essential for debugging API integration issues
Sources	Set breakpoints and step through actual JavaScript execution line by line
Application	Inspect localStorage/sessionStorage/cookies, Service Workers, and cached data
Performance	Record and analyze a timeline of rendering/scripting/painting work, to find real performance bottlenecks

Real-time example: Debugging "why is my API call returning 401" starts in the Network tab — inspecting the actual request headers reveals whether the Authorization header was sent at all, and whether it was correctly formatted, far faster than guessing from application code alone.

51. Debugging React with React DevTools

The **React Developer Tools** browser extension adds two dedicated tabs specifically for React apps:

Tab	Purpose
Components	Inspect the live component tree, view/edit any component's current props and state values directly, and see exactly which component a piece of rendered UI belongs to
Profiler	Record a render session and see exactly WHICH components re-rendered, how long each took, and WHY (which prop/state triggered it) — the primary real tool for diagnosing actual React performance problems

The Profiler tab's "why did this render" feature is invaluable for real-world performance debugging — instead of guessing whether `React.memo` (Section 22) is actually helping, you can see concretely whether a component re-rendered unnecessarily and trace it to the exact prop/state change that caused it.

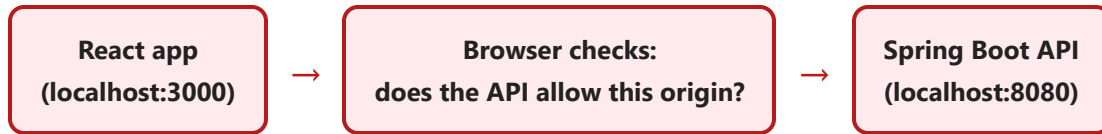
52. HTTP & HTTPS Deep Dive

Concept	Explanation
HTTP	The application-layer protocol browsers and servers use to exchange requests/responses (methods, headers, status codes) — covered in full depth in the API Development guide
HTTPS	HTTP layered on top of TLS/SSL encryption — encrypts the entire request/response in transit, preventing eavesdropping or tampering by anyone between the client and server
TLS Handshake (simplified)	Client and server exchange certificates/keys to establish a shared encryption key BEFORE any actual HTTP data is sent, authenticating the server's identity via a trusted Certificate Authority

Why HTTPS is mandatory today, not optional: Modern browsers actively block or heavily restrict powerful APIs (geolocation, camera/microphone access, Service Workers) on plain HTTP pages, and clearly mark HTTP sites as "Not Secure" — HTTPS is a hard requirement for any real production frontend, not just a nice-to-have.

53. CORS Deep Dive

CORS (Cross-Origin Resource Sharing) is a BROWSER-enforced security rule blocking a web page on one origin (domain+port+protocol) from calling an API on a DIFFERENT origin, unless the server explicitly allows it via response headers.



```
// Spring Boot – allowing the React dev server's origin
@CrossOrigin(origins = "http://localhost:3000")
@RestController
public class ProductController { ... }
```

CORS Preflight Requests

For "non-simple" requests (JSON bodies, custom headers like Authorization, methods like PUT/DELETE), the browser first automatically sends an `OPTIONS` "preflight" request asking the server "do you allow this exact method/headers from my origin?" — only if the server responds allowing it does the browser then send the REAL request. This preflight is entirely automatic and invisible in your code, but very visible in the Network tab (Section 50) when debugging a CORS issue.

ISSUE Console error: "has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present."

FIX The backend hasn't been configured to allow the frontend's exact origin — add a CORS configuration (global bean or `@CrossOrigin`) on the Spring Boot backend explicitly listing the frontend's origin(s); this can NEVER be fixed from the frontend side alone, since it's the SERVER's response headers that grant permission.

ISSUE CORS works for GET requests but fails specifically for POST/PUT requests with a JSON body.

FIX The `OPTIONS` preflight request for the non-simple POST/PUT is likely being blocked or not handled correctly by the backend — ensure the server explicitly allows the needed HTTP methods and the `Content-Type` / `Authorization` headers in its CORS configuration, not just the origin.

54. Production Deployment — Where & How Frontend Apps Are Deployed

A React app's production "build" (`npm run build`) produces a folder of STATIC files (HTML, JS, CSS, images) — no Node.js server is needed to actually SERVE a plain CSR React app; it just needs any web server or CDN capable of serving static files.

Deployment Option	How It Works	Best For
Vercel / Netlify	Git-connected platforms that auto-build and deploy on every push, with a global CDN and free SSL built in	Fastest path to production for most React/Next.js apps — the modern default choice for many teams
Nginx (self-hosted / on a VM)	A lightweight web server configured to serve the static build folder, often also acting as a reverse proxy to the backend API	Teams needing full infrastructure control, or deploying alongside Spring Boot on the same VM/server
Docker + Kubernetes (Nginx-based image)	The static build is baked into an Nginx-based Docker image, deployed as a Kubernetes pod alongside backend microservices	Organizations already running a full Kubernetes-based microservices platform (exactly the pattern in the Docker & Kubernetes guide)
Azure Static Web Apps / AWS S3 + CloudFront	Cloud-native static hosting directly integrated with a CDN for global edge caching	Teams already standardized on Azure or AWS, wanting managed, scalable static hosting with minimal ops overhead

Key architectural point: The React frontend and the Spring Boot backend are almost always deployed as TWO SEPARATE, independently-scaled units in production — the frontend as static files behind a CDN, the backend as a running API service (Docker/Kubernetes/App Service) — exactly the separated deployment model covered in the React + Spring Boot API Integration guide.

55. Deployment Deep Dive — Nginx, Docker, Vercel/Netlify, Azure, S3+CloudFront

```
# Build the production-ready static files
npm run build          # outputs to /dist (Vite) or /build (CRA)

# Nginx config serving a React SPA (with correct SPA fallback routing!)
server {
    listen 80;
    root /usr/share/nginx/html;
    location / {
        try_files $uri /index.html; # CRITICAL for SPAs – see the issue below
    }
    location /api/ {
        proxy_pass http://backend:8080/api/; # reverse proxy to the Spring Boot backend
    }
}

# Dockerfile – multi-stage build (Docker & Kubernetes guide, Section 6-7 pattern)
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

ISSUE Refreshing the browser on a client-side route like `/products/42` returns a 404 Not Found from the web server, even though the same URL works fine when navigated to via an in-app `<Link>` click.

FIX The web server is looking for a REAL file/folder at `/products/42`, which doesn't exist — it only exists as a CLIENT-SIDE React Router route. Configure the server to fall back to serving `index.html` for any unmatched path (the `try_files $uri /index.html;` line above), letting React Router handle the routing once the app loads.

56. CI/CD for React Apps

```
# Example GitHub Actions workflow for a React app
name: Deploy React App
on: { push: { branches: [main] } }
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm run test -- --run
      - run: npm run build
      - name: Deploy to hosting
        run: # deploy /dist to Vercel/Netlify/S3/Azure Static Web Apps
```

A real CI/CD pipeline for a React app mirrors the same principles covered in the dedicated CI/CD guide — install dependencies with `npm ci` (not `npm install`) for reproducibility, run tests and linting as a quality gate BEFORE building, then deploy the static build output, often to a staging environment first with production requiring an approval gate.

57. Environment Variables & Config Per Environment

```
# .env.development
VITE_API_URL=http://localhost:8080/api

# .env.production
VITE_API_URL=https://api.myproductionapp.com/api

// Usage in code:
const apiUrl = import.meta.env.VITE_API_URL; // Vite-specific prefix; CRA instead requires
REACT_APP_ prefix
```

ISSUE An environment variable is `undefined` inside the running React app, despite being correctly set in the `.env` file.

FIX Build tools ONLY expose environment variables prefixed with a specific keyword (`VITE_` for Vite, `REACT_APP_` for CRA) to client-side code, as a deliberate safety measure preventing accidental exposure of server-only secrets — rename the variable with the correct required prefix.

Critical security reminder: ANYTHING placed in a frontend environment variable is bundled directly into the publicly downloadable JavaScript files and is fully visible to anyone using browser DevTools — NEVER put real secrets (database passwords, private API keys) in frontend environment variables; they belong only in the backend.

58. Common React Bugs, Issues & Fixes (Extended)

ISSUE "Too many re-renders" error, crashing the app.

FIX A state setter is being CALLED directly in the render body (e.g., `setCount(count+1)` outside an event handler/effect) instead of being passed as a reference — this creates an infinite render loop; only call setters inside event handlers or `useEffect`.

ISSUE A child component doesn't update even though its prop value visibly changed in the parent.

FIX Check whether the child is wrapped in `React.memo` and receiving a NEW object/array/function reference every render that looks different by reference even if "equal" in content — either memoize the prop with `useMemo/useCallback` in the parent, or provide a custom comparison function to `React.memo`.

ISSUE Console warning: "Each child in a list should have a unique key prop."

FIX Add a stable, unique `key` (ideally a database ID, never the array index for reorderable lists) to the top-level element returned inside a `.map()` call (Section 16).

ISSUE "Can't perform a React state update on an unmounted component" warning.

FIX An async operation (fetch, timer) resolves AFTER the component has already unmounted and tries to call a state setter — add a cleanup flag/`AbortController` in `useEffect`'s cleanup function to cancel or ignore the update if the component is gone (Section 27, 36).

ISSUE Form input feels "laggy" or loses focus while typing in a large, complex form.

FIX Likely the ENTIRE large form re-renders on every keystroke because state lives too high up the tree — split the form into smaller, independently-memoized sub-components, or move rapidly-changing local field state down closer to where it's actually used.

ISSUE Redux state updates, but the connected component doesn't re-render.

FIX The reducer likely mutated state WITHOUT going through Redux Toolkit's Immer-powered `createSlice` (e.g., in older/manual Redux code), so the store's top-level reference didn't actually change, and `useSelector` never detects a difference — always use `createSlice` or return a genuinely new object/array from plain reducers.

ISSUE An app works perfectly in development but shows a blank white screen in production after deployment.

FIX Commonly a routing/base-path misconfiguration (the server isn't serving `index.html` as a fallback, Section 55), a missing/incorrect environment variable for the production API URL, or an uncaught error with no Error Boundary (Section 21) — check the Console and Network tabs (Section 50) in production first.

59. Common Production Issues & Fixes

ISSUE The production bundle is huge (several MB), causing slow initial load especially on mobile networks.

FIX Use code-splitting via dynamic `import()` and `React.lazy / Suspense` to split large routes/rarely-used components into separate chunks loaded on demand, and analyze the bundle (e.g., with `vite-bundle-visualizer`) to find unexpectedly large dependencies.

ISSUE Users see a stale, cached version of the app after a new deployment, even after a hard refresh.

FIX Ensure the build tool outputs content-hashed filenames (e.g., `app.a1b2c3.js`) so browsers/CDNs correctly treat a new deployment as genuinely new files, and set appropriately short cache headers specifically on `index.html` (which references those hashed files) while long-caching the hashed assets themselves.

ISSUE API calls that work from a developer's machine fail with CORS or network errors once deployed to production.

FIX The production frontend's real domain likely isn't included in the backend's CORS allowed-origins list (Section 53), or the production `VITE_API_URL` environment variable wasn't correctly set at BUILD time for that environment (Section 57).

ISSUE A security scan flags that a JWT or API key is visible in the deployed JavaScript bundle.

FIX Any value placed in a frontend environment variable or hardcoded in source is fully visible in the shipped bundle — genuine secrets must live only on the backend; the frontend should only ever hold short-lived, user-scoped tokens (Section 44), never long-lived API keys/secrets.

60. React Interview Questions & Answers (40 Q&A)

Q1. Why did React come into existence, and what problem did it primarily solve?

A: It replaced slow, error-prone, manual/imperative DOM manipulation (common with jQuery) with a declarative model — describing what the UI should look like for given data, letting React efficiently compute and apply the minimal real DOM changes via the Virtual DOM.

Q2. What is the Virtual DOM, and why does it improve performance?

A: An in-memory JavaScript representation of the UI that React can diff cheaply; comparing two Virtual DOM trees and applying only the ACTUAL differences to the real DOM is far faster than naively re-rendering the entire real DOM on every change.

Q3. What's the difference between SPA and MPA?

A: An MPA reloads a full new HTML page from the server on every navigation; an SPA loads one HTML shell once and uses JavaScript (React Router) to swap content in place without a full reload, giving a faster, app-like navigation experience after the initial load.

Q4. Explain CSR vs SSR vs SSG.

A: CSR renders content in the browser using JavaScript after an initial near-empty HTML shell; SSR renders full HTML on the server per-request before sending it, then "hydrates" it client-side; SSG pre-renders HTML once at build time, served as static files thereafter — each trades off initial load speed, SEO, and hosting simplicity differently.

Q5. Why might a team choose React over Angular?

A: A gentler learning curve (small core API vs Angular's DI/RxJS/module system), more flexibility in choosing router/state-management libraries, a larger ecosystem/hiring pool, and JSX often feeling closer to plain JavaScript than Angular's separate template syntax — though Angular remains a strong, valid choice for teams wanting an opinionated, all-in-one framework.

Q6. What is JSX, and is it valid JavaScript?

A: JSX is HTML-like syntax embedded in JavaScript that Babel compiles down to `React.createElement()` calls — it's not valid JavaScript on its own and requires a build step to transform it before running in a browser.

Q7. What's the difference between props and state?

A: Props are passed INTO a component from its parent and are read-only from the child's perspective; state is owned and managed BY the component itself and can change over time, triggering re-renders when updated via its setter.

Q8. Why shouldn't you use array index as a key in a dynamic list?

A: If the list can reorder, filter, or have items inserted/removed from the middle, index-based keys cause React's reconciliation to misidentify which DOM element corresponds to which data item, leading to bugs like input values or component state "sticking" to the wrong row.

Q9. What are the Rules of Hooks, and why do they exist?

A: Only call Hooks at the top level (never in loops/conditions) and only from React components or custom Hooks — React relies on Hooks being called in the exact same order every render to correctly match each call to its own internal state slot.

Q10. What does the `useEffect` dependency array control?

A: It determines WHEN the effect re-runs — omitted means every render, an empty array means only once on mount, and an array with values means it re-runs whenever any of those specific values change between renders.

Q11. What is prop drilling, and how does Context solve it?

A: Prop drilling is passing a value through many intermediate components that don't use it themselves, just to reach a deeply nested child that does; Context lets any descendant read a provided value directly, skipping the intermediate layers entirely.

Q12. When would you choose Redux Toolkit over the Context API?

A: When shared state changes frequently and is used broadly across the app — Context can cause every consuming component to re-render on any change, while Redux's `useSelector` lets components subscribe to only the specific slice of state they actually need.

Q13. What is the difference between `useMemo` and `useCallback`?

A: `useMemo` memoizes a computed VALUE (avoiding an expensive recalculation); `useCallback` memoizes a FUNCTION reference itself (useful for stable references passed to memoized children or effect dependencies) — `useCallback` is essentially `useMemo` specialized for functions.

Q14. How would you debug a component that seems to re-render too often?

A: Use React DevTools' Profiler tab to record a session and see exactly which components re-rendered and why — commonly caused by new object/array/function references being passed as props on every parent render, fixable with `useMemo`/`useCallback` and `React.memo`.

Q15. What is an Error Boundary, and what can't it catch?

A: A class component that catches JavaScript errors thrown during rendering in its child tree, showing a fallback UI. It cannot catch errors in event handlers, asynchronous code, or during server-side rendering — those still need regular try/catch handling.

Q16. Why does `fetch()` require manually checking `response.ok`, unlike Axios?

A: `fetch()` only rejects on genuine network-level failures; an HTTP error status code like 404/500 is still considered a "successful" network exchange from `fetch`'s perspective, so `response.ok` must be checked manually — Axios automatically throws for non-2xx responses instead.

Q17. What is the purpose of Axios interceptors?

A: They let you handle cross-cutting API concerns — attaching an auth token to every outgoing request, or globally redirecting to login on a 401 response — in one centralized place, instead of repeating that logic in every component making API calls.

Q18. Explain the Redux data flow in one sentence.

A: A component dispatches an action describing what happened, a pure reducer function computes the new state based on that action, the store updates, and every subscribed component re-renders with the new relevant state.

Q19. Why does Redux Toolkit let you write reducer code that looks like direct mutation?

A: It uses the Immer library internally, which lets you write mutation-looking code while actually producing a correct, brand-new immutable state object behind the scenes — removing classic Redux's tedious, error-prone manual immutable-update boilerplate.

Q20. How does a React app implement JWT-based authentication end to end?

A: On login, the backend returns a signed JWT; the frontend stores it (localStorage, memory, or an HttpOnly cookie) and attaches it as an Authorization Bearer header on every subsequent API call (often via an Axios interceptor), and the backend independently validates the token's signature/expiry on each request.

Q21. Why is a frontend "protected route" not real security on its own?

A: It only hides a page/component visually within the app — a user can still call the underlying API directly (e.g., via Postman) bypassing the frontend entirely; the backend must independently enforce real authorization on every endpoint.

Q22. What's the trade-off between storing a JWT in localStorage vs an HttpOnly cookie?

A: localStorage is simple but readable by any JavaScript on the page, making it vulnerable if an XSS attack occurs anywhere in the app; an HttpOnly cookie can't be read by JavaScript at all (immune to XSS token theft) but requires CSRF protection and careful cross-origin cookie configuration.

Q23. What is npm ci, and why is it preferred in CI/CD pipelines over npm install?

A: npm ci deletes node_modules first and installs the EXACT versions pinned in package-lock.json, failing if it's out of sync with package.json — this strictness guarantees a reproducible, identical dependency tree on every build, unlike npm install which can subtly drift.

Q24. Explain semantic versioning's ^ and ~ symbols in package.json.

A: ^1.4.0 allows any MINOR or PATCH update within the same major version (e.g., up to but not including 2.0.0); ~1.4.0 allows only PATCH updates within the same minor version (e.g., up to but not including 1.5.0) — controlling how much automatic version drift is acceptable for a dependency.

Q25. Why is Vite generally faster than Webpack-based tools like Create React App during development?

A: Vite serves source files directly over native browser ES modules without bundling the whole app upfront, only transforming files on demand as the browser requests them — Webpack-based tools must bundle the entire application before the dev server can serve anything, which gets slower as the app grows.

Q26. What does Hot Module Replacement (HMR) actually do?

A: It updates only the specific changed module in the running browser in place, preserving the current application state (like form inputs or open modals), instead of triggering a full page reload that would reset everything.

Q27. How would you debug a failed API call using browser DevTools?

A: Open the Network tab, find the specific failed request, and inspect its status code, request headers (was the auth token actually sent?), and response body — this reveals far more concrete information than guessing from application-level console logs alone.

Q28. What is CORS, and which side (frontend or backend) actually fixes a CORS error?

A: CORS is a browser-enforced rule blocking cross-origin requests unless the server explicitly allows them via response headers — it can ONLY be fixed on the backend/server side, since the browser is checking the SERVER's response headers, not anything the frontend can control.

Q29. What is a CORS preflight request?

A: An automatic OPTIONS request the browser sends before certain "non-simple" cross-origin requests (custom headers, methods like PUT/DELETE, JSON content-type) to ask the server whether the actual request is permitted, before sending it for real.

Q30. Why must a React SPA's web server be configured with a fallback to index.html?

A: Client-side routes (like /products/42) only exist within React Router's in-browser routing, not as real files/folders on the server — without a fallback, refreshing or directly visiting such a URL causes the web server to correctly report a genuine 404, since no matching file exists.

Q31. Why should you never put real secrets in a frontend environment variable?

A: Frontend environment variables get bundled directly into the publicly downloadable JavaScript files, fully visible to anyone using browser DevTools — genuine secrets (database passwords, private API keys) must remain backend-only.

Q32. What is code-splitting, and why does it matter for production performance?

A: Using dynamic import() with React.lazy/Suspense to split the app into smaller chunks loaded on demand (e.g., per route), rather than shipping one giant JavaScript bundle upfront — significantly improving initial load time, especially on slower mobile networks.

Q33. Why do deployed React apps use content-hashed filenames for their JS/CSS bundles?

A: So browsers and CDNs can safely cache these files for a very long time (since a genuinely new deployment produces a NEW filename), while index.html itself (which references the current hashed filenames) is served with short/no caching, ensuring users always get the latest version without stale-cache issues.

Q34. What's the difference between useState and useReducer, and when would you prefer the latter?

A: useState suits independent, simple state values; useReducer is preferred once state logic involves multiple related sub-values updated together via distinct named "actions," centralizing the transition logic in one reducer function similar to a mini-Redux pattern.

Q35. What is a custom Hook, and what problem does it solve?

A: A reusable function (starting with "use") that encapsulates stateful logic (like data fetching) so multiple components can share the same behavior without duplicating code — the modern replacement for the older Higher-Order Component and Render Props patterns.

Q36. Explain React's reconciliation process in your own words.

A: After a state/props change causes a component to re-render, React builds a new Virtual DOM tree, compares (diffs) it against the previous tree using a heuristic algorithm, and applies only the minimal set of actual changes to the real browser DOM — avoiding expensive, unnecessary full DOM rebuilds.

Q37. How does React Router enable navigation without full page reloads?

A: It intercepts clicks on its own <Link> components and uses the browser's native History API to update the URL bar without triggering an actual server request/page reload, then re-renders only the component tree matching the new path.

Q38. Why is it important to clean up subscriptions/timers/event listeners inside useEffect?

A: Without returning a cleanup function, subscriptions or timers keep running (and holding references) even after a component unmounts, causing memory leaks and "state update on unmounted component" warnings — the returned cleanup function runs before the next effect and on unmount specifically to prevent this.

Q39. What real production issue have you seen from deploying a React app, and how would you resolve it?

A: A blank white screen in production while everything worked in development — typically traced to a missing SPA fallback route on the web server, an incorrect production API URL environment variable, or an uncaught rendering error with no Error Boundary in place; checking the browser Console/Network tabs in production is the first diagnostic step.

Q40. How would you explain the overall React + Spring Boot deployment architecture to a new team member?

A: The React frontend is built into static files and deployed independently (behind a CDN, Nginx, or a static

hosting service), while the Spring Boot backend runs as a separately deployed, independently-scaled API service (often containerized) — the two communicate purely over HTTP/JSON, with CORS configured on the backend to allow the frontend's specific production domain.

61. Quick Revision Summary

- React replaced manual, imperative DOM manipulation with a declarative, Virtual-DOM-diffing model.
- SPA (client-side routing, no full reloads) vs MPA (full page reload per navigation); CSR/SSR/SSG trade off initial load speed, SEO, and hosting simplicity differently.
- React favors a small core + flexible ecosystem choices; Angular is a full, opinionated framework — both are valid, context-dependent choices.
- Components are functions of props/state; Hooks (useState, useEffect, custom Hooks) manage state and side effects in functional components.
- Always use a stable, data-based key for dynamic lists — never the array index for reorderable lists.
- React Router enables client-side navigation; protected routes are UX only — the backend must enforce real authorization.
- Axios adds interceptors and automatic error-throwing over native fetch; centralize API calls and auth-token attachment in one client.
- Redux Toolkit centralizes shared app state via slices/reducers/actions, using Immer to allow mutation-looking (but actually immutable) reducer code.
- CORS errors are fixed only on the backend; the SPA web server needs an index.html fallback for client-side routes to survive a refresh.
- Never place real secrets in frontend environment variables — they're fully visible in the shipped JavaScript bundle.